—

Questão 1. Observe o código abaixo:

```c
#include <stdio.h>

int main() {
    int numero = 10;
    int *p = &numero;

    printf("numero = %d\n", numero);
    printf("p = %p\n", (void*)p);
    printf("*p = %d\n", *p);

    return 0;
}
```

a) O que está armazenado na variável numero?

10

b) O que está armazenado na variável p?

A variavel p e um ponteiro q esta indicando o endereço da variavel

numero

c) O que significa *p?

Significa que a variavel criada e do tipo ponteiro que guarda o endereço de memoria ao inves de um dado

d) O que será apresentado pelo printf("*p = %d\n", *p)?

mostra o endereço de um espaço na memoria que o ponteiro encontrou para guardar o valor da variavel

e) Explique detalhadamente o que acontece se substituirmos:

*p = 50;

o ponteiro vai subistituir o valor que esta dentro da variavel por 50

Questão 2. Qual é a diferença entre os seguintes tipos?
Crie um programa que:

1. Declare uma variável int numero = 100.
2. Crie um ponteiro para essa variável.
3. Mostre o valor de numero.
4. Altere o valor de numero para 500 utilizando **somente o ponteiro**.
5. Mostre novamente o valor de numero.

Resultado esperado:
Antes: 100
Depois: 500

```
#include <stdio.h>
#include<stdlib.h>
int main()
{

int numero = 100;
int*n1 = &numero;

printf("valor do numero:%d\n",numero);

*n1 = 500;
printf("valor do numero:%d",numero);


return 0;
}
```

Questão 3. O programa utiliza dois ponteiros, um para cada variável (a e b). Primeiro, são exibidos os valores e os endereços de memória das variáveis. Em seguida, os valores são alterados por meio dos ponteiros, passando a para 100 e b para 200.

```
#include <stdio.h>

int main() {

   int a = 10, b = 20;

    int *p1 = &a;
```

```
    int *p2 = &b;

    printf("\n *** Valor inicial ***\n");

    printf("a: valor = %d, endereco = %p\n", *p1, (void*)p1);

    printf("b: valor = %d, endereco = %p\n", *p2, (void*)p2);

    *p1 = 100;

    *p2 = 200;

    printf("\n*** Apos Alteração ***\n");

    printf("a: novo valor = %d\n", a);

    printf("b: novo valor = %d\n", b);

    return 0;

}
```

Questã 4. Observe o código abaixo e responda as questões:

```c
int *numero;

numero = malloc(sizeof(int));

free(numero);
```

a) O que é numero?

O ponteiro

b) O que malloc(sizeof(int)) está fazendo, explique detalhadamente esse código?

O numero e um ponteiro que vai indicar um espaço de armazenamento na memoria nesse caso o espaco necessario n foi estipulado ao inves disso vamos definir uma alocacao dinamica o malloc serve para solicitar um espaco na memoria no valor de byte de um int

c) Em qual área da memória a variável **numero** fica armazenada?

pilha

d) Onde fica o espaço de memória reservado pelo malloc?

heap

e) Para que serve o free()?

Para liberar a memoria que foi reservada pelo malloc

Questão 5. Crie um programa que reserve dinamicamente espaço para **um inteiro**.

Depois:

1. Armazene o valor 100.
2. Mostre o valor armazenado.
3. Mostre o endereço da memória alocada.
4. Libere a memória utilizando free().

```
#include <stdio.h>
#include <stdlib.h>

int main() {

 int *valor = (int *) malloc(sizeof(int));

 *valor = 100;

 printf("Valor armazenado: %d\n", *valor);
 printf("Endereço de memória: %p\n", (void *)valor);

 free(valor);

 return 0;

}
```

Questão 06 – Crie um programa em C que utilize um ponteiro para armazenar **10 números inteiros**.

Utilize a declaração:

int *numeros;

O programa deverá:

1. **Alocar memória** suficiente para armazenar 10 números inteiros utilizando malloc.
2. **Ler** 10 números digitados pelo usuário.
3. **Armazenar** os números na memória alocada.
4. **Exibir** todos os números armazenados.
5. **Liberar** a memória utilizada com free()

```
#include <stdio.h>
#include <stdlib.h>

int main() {

 int *num;
 num = (int *) malloc(10 * sizeof(int));

 printf("Digite 10 números inteiros:\n");
 for (int i = 0; i < 10; i++) {

 printf("Número %d: ", i + 1);
 scanf("%d", &num[i]);
 }

 printf("\nNúmeros armazenados:\n");

 for (int i = 0; i < 10; i++) {
 printf("%d ", num[i]);
 }
 free(num);

 return 0;
}
```

Questão 07 – Explique o que significa:

malloc(10 * sizeof(int));

o malloc serve para reservar espaço na memoria o sizeof conta quantos bytes um int ocupa na memoria e multiplica por 10 para calcular o valor total necessario

Depois responda:

a) Quantos int estão sendo reservados?
10

b) Por que usamos sizeof(int)?

para calcular o valor exato em bytes que o int ocupa

c) A quantidade de bytes reservados é necessariamente 10?

Nao,cada int ocupa 4 bytes o valor total reservado e 40 bytes

d) Se sizeof(int) for 4 bytes, quantos bytes serão reservados?

40 bytes

Questão 08 – Crie um programa em C que declare uma variável inteira com o valor **10** e, em seguida, crie um ponteiro que aponte para essa variável.

O programa deverá mostrar o valor da variável **antes da alteração** e, utilizando o ponteiro, alterar seu valor para **50**. Ao final, exiba novamente o valor da variável, mostrando o resultado após a alteração.

```
#include <stdio.h>

int main() {

 int valor = 10;

 int *p = &valor;

 printf("Valor antes da alteracao: %d\n", valor);

 *p = 50;

 printf("Valor apos a alteracao: %d\n", valor);

 return 0;

}
```

Questão 09 – Faça um programa que leia dois números inteiros.

Crie dois ponteiros, um para cada número.

Utilizando os ponteiros, mostre:

- o valor dos dois números;
- o endereço de memória de cada número;
- a soma dos dois números por meio do ponteiro.

```
#include <stdio.h>

int main() {

    int num1, num2;

    int *p1, *p2;

    printf("Digite um numero inteiro: ");
    scanf("%d", &num1);

    printf("Digite outro numero inteiro: ");
    scanf("%d", &num2);

    p1 = &num1;
    p2 = &num2;


    printf("\n*** Resultados ***\n");
    printf("Valor do primeiro numero: %d\n", *p1);
    printf("Valor do segundo numero: %d\n", *p2);

    printf("Endereco do primeiro numero: %p\n", (void*)p1);
    printf("Endereco do segundo numero: %p\n", (void*)p2);

    int soma = *p1 + *p2;
    printf("Soma dos dois numeros : %d\n", soma);

    return 0;
}
```

Questão 10 – Crie um programa em C que leia **dois números inteiros** informados pelo usuário.

Em seguida, crie uma função chamada trocar, que receba como parâmetros os **endereços de memória dos dois números** e utilize ponteiros para trocar os valores entre eles.

Ao final, mostre os valores das duas variáveis **antes e depois da troca**, permitindo verificar o resultado da operação.

DESAFIADORA – Crie um programa em C que utilize **alocação dinâmica de memória** com malloc() e realloc().

O programa deverá:

1. Criar um ponteiro int *numeros.
2. Utilizar malloc() para reservar espaço para **5 números inteiros**.
3. Verificar se a memória foi alocada corretamente.
4. Armazenar os valores de **1 a 5** na memória alocada.
5. Utilizar realloc() para aumentar o espaço e permitir o armazenamento de **10 números inteiros**.
6. Preencher as novas posições com os valores de **6 a 10**.
7. Exibir os **10 números** na tela.
8. Ao final, utilizar free() para liberar a memória.

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *num;

    num = (int *) malloc(5 * sizeof(int));

    for (int i = 0; i < 5; i++) {
        num[i] = i + 1;
    }

    num = (int *) realloc(num, 10 * sizeof(int));

    for (int i = 5; i < 10; i++) {
        num[i] = i + 1;
    }

    printf("Os 10 numeros alocados sao:\n");
```

```
    for (int i = 0; i < 10; i++) {
        printf("%d ", num[i]);
    }
    printf("\n");

    free(num);

    return 0;
}
```